

Protocol Anonymous Verification Mechanism for Computing tasks AVC

For each verifier V_i to verify the computation $\tau : \{\sigma_0, \sigma_1, \dots, \sigma_n\}$

★ Initialization phase

▷ Setup(1^λ) $\rightarrow (pp, st)$

- initialize the verifier-membership Merkle tree $\mathcal{MKT}$, the verifier list $\mathcal{VL}_\tau$, and the nullifier set $\mathcal{DS}_\tau$;
- receive snapshot commitments $\{\sigma\}$ and $H_1(\text{Ser}(\sigma_{\text{gas}}, r))$ from the executor;
- set the basic stake amount v_{stake} for task τ ;
- generate a task-specific public key opk and derive the staking address $\mathcal{A}_\tau = H_1(opk)$;
- run NIZK.KeyGen to obtain proving and verification keys (zpk, zvk) ;
- set public parameters $pp = (\mathcal{A}_\tau, zpk, zvk, v_{\text{stake}})$ and state $st = (\mathcal{MKT}, \mathcal{VL}_\tau, \mathcal{DS}_\tau)$.

★ Staking and Registration phase

◇ Each verifier V_i generates system parameters

- V_i obtains funds via anonymous mixers;
- AnonyReg($1^\lambda, pp$);
- Verify the proof generated during registration
- ensure the note C_i is valid;
- if verification passes then $\mathcal{VL}_\tau.append(C_i)$.

◇ Each verifier V_i stakes to participate in the election

- interpret st as $(\mathcal{MKT}, \mathcal{VL}_\tau, \mathcal{DS}_\tau)$;
- AnonyStake($C_i, pp, UKey_i$);
- check $N_i \notin \mathcal{DS}_\tau$; if violated, exit.
// reject double-spending by unique tag N_i
- $\mathcal{DS}_\tau.append(N_i)$;
- check the conservation equation

$$C_i + \Delta C \stackrel{?}{=} C_{\text{stake}} + C_{\text{change}}$$

(or equivalently $C_i \stackrel{?}{=} C_{\text{stake}} + C_{\text{change}} + \Delta C$);

// Pedersen-commitment value conservation

- check $C_{\text{stake}} \stackrel{?}{=} C_{\text{receipt}} + v_{\text{stake}}H$;
- deposit pledged funds into the system pool;
- $\mathcal{VL}_\tau.append(C_{\text{stake}})$;
- regard C_{stake} as the new current note of V_i .

★ Verification and Scheduling phase

◇ Schedule verifiers to verify computation tasks τ

- sample R from the random beacon;
- parse $st.\mathcal{VL}_\tau$ as the candidate set $\{C_{\text{stake}}\}$;
// screen verifier members
- $\mathcal{VL}'_\tau \leftarrow \text{Filter}(\mathcal{VL}_\tau, R)$;
- compute Merkle-tree commitment H_{root} over $\mathcal{VL}'_\tau$;
- output $\mathcal{VL}'_\tau$.

◇ For each verifier $V_i \in \mathcal{VL}'_\tau$ to verify task τ

- verify assigned snapshot segments following the multi-round procedure;
- Endorse($pp, st, C_{\text{stake}}, H_{\text{root}}$)
 - authenticate anonymous identity via ZK proofs;
 - verify Merkle membership and linkability
 - output 1 only if both checks succeed.

• If authentication succeeds:

Res.ver is err:

- submit the erroneous snapshot σ_i on-chain;
- blockchain executes $Ver(\sigma_i, \sigma_{i+1})$;
- reward the detecting verifier and slash the executor.

Res.ver is true:

- record verification effort Res.Gas_i ;
- publish $(\text{Ser}\{\sigma_k, \dots, \sigma_t\}, r)$;
- check consistency with $\tau.\text{Gas}$ and reward or slash accordingly.